LAUNCH FILES + PARAMETERS (YAML) — THE PRACTICAL WAY

How to run multi-node systems and tune behavior (SLAM/Nav2 later)

UOttawa Mobile Robotics • Instructor: Ali Karimzadeh

uOttawa

Why parameters + launch files matter

Most real ROS systems are **configured**, not hard-coded.

- **Parameters** = settings (numbers/strings/flags) that change node behavior
- **Launch** = start many nodes + set params + remap topics + namespaces
- SLAM/Nav2 are mainly: **launch + config YAML** (lots of parameters)
- Goal: you can reproduce runs reliably (same setup every time)
- Docs: Parameters • Launch

Parameters: what they are (and what they are NOT)

A parameter is a named value stored **inside a node**.

- Good for: `max_speed`, `use_sim_time`, `map_frame`, thresholds, gains…
- Not good for: high-rate sensor data → that belongs in **topics**
- Parameters can be set at **startup** (recommended) or **runtime** (if node allows)
- Types: bool, int, float, string, arrays (YAML-typed)
- Key idea: Node behavior = **code + parameter values**

The `ros2 param` toolbox (daily commands)

First: find the node name. Then inspect its parameters.

- ```
  ros2 node list            # see running nodes
  ros2 param list           # list parameters of ALL nodes
  ```
- Get/describe a specific parameter:
- ```
  ros2 param get /node_name use_sim_time
  ros2 param describe /node_name use_sim_time
  ```
- Official how-to: [Using ros2 param]

uOttawa

Set parameters (runtime) + dump to YAML (snapshot)

Runtime changes work only if the node allows changing that parameter.

- Set a value (YAML typing matters):
- ```
  ros2 param set /node_name use_sim_time false
  ros2 param set /node_name max_speed 0.25
  ```
- Dump current params to YAML (shareable config):
- ```
  ros2 param dump /node_name > dumped_params.yaml
  ```
- Tip: If `set` fails, the parameter may be **read-only** or **not declared**.

uOttawa

Quiz: Parameters are stored where?

In ROS 2, parameters are primarily stored…

- **A** In a global server like ROS 1 `roscore`
- **B** Inside each node (per-node parameters)
- **C** Inside RViz
- **D** Inside `/tf` transforms

uOttawa

YAML parameter files (the standard way for big configs)

YAML = a readable config format (indentation matters).

- Why YAML: many parameters → easier than 200 CLI `param set` commands
- Rule: use **spaces**, not tabs
- In ROS 2 YAML, parameters go under the key: `ros__parameters`
- You address parameters by **node name** (and optional namespace)
- Guide: [Migrating YAML param files]

YAML structure (general example)

This file sets params for one node + one global param for all nodes.

- ```
  # save as: config/demo_params.yaml
  ```
- ```
  /my_node:
    ros__parameters:
      use_sim_time: false
      max_speed: 0.25
      frame_id: "base_link"

  /**:
    ros__parameters:
      debug: true
  ```
- Notes: `/**` is a wildcard → applies to **any node** in any namespace.

uOttawa

Load a YAML params file when starting a node (recommended)

Startup params are reproducible (best for SLAM/Nav2 configs).

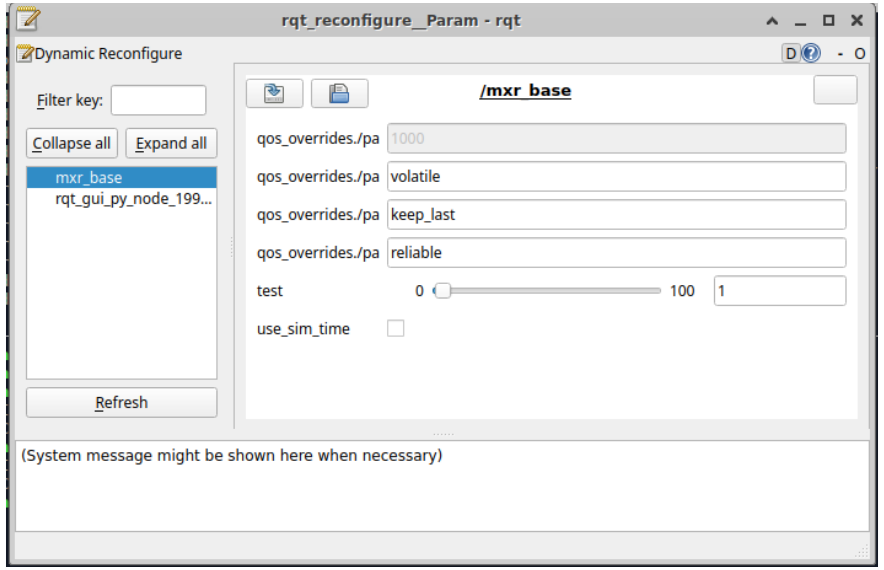
- General pattern:
- ```
  ros2 run <pkg> <exe> \
    --ros-args --params-file /path/to/demo_params.yaml
  ```
- Example (if you have a node named `my_node` in your package):
- ```
  ros2 run my_pkg my_node --ros-args --params-file config/demo_params.yaml
  ```
- ROS args design: [ROS command line arguments]

uOttawa

GUI tuning: `rqt_reconfigure` (live parameter editing)

Use this when you want to **tune** and see immediate effects.

- Install (once):
- ```
  sudo apt update
  sudo apt install -y ros-humble-rqt-reconfigure
  ```
- Run:
- ```
  rqt
  ```
- In rqt: `Plugins → Configure → Dynamic Reconfigure`
- If plugin not shown: `rqt --force-discover`
- Docs: [rqt_reconfigure]



Source: ROS Answers archive — rqt_reconfigure screenshot

uOttawa

A practical tuning workflow (do this in projects)

Tune → test → dump → commit the YAML file.

- 1) Start system with a baseline config (launch/YAML)
- 2) Tune key params live (CLI or `rqt_reconfigure`)
- 3) Save a snapshot:
- ```
  ros2 param dump /node_name > tuned.yaml
  ```
- 4) Put final configs in your package: `config/` folder

uOttawa
  ```

Launch files: what they do

Launch files start and configure **multiple nodes** together.

- Start nodes, set parameters, remap names, set namespaces
- Make your system reproducible: **one command** to run everything
- You will read many `.launch.py` files in TB3/SLAM/Nav2 stacks
- Launch tutorial: Launch main

Launch file formats (3 types)

ROS 2 supports multiple formats — you will mostly see Python.

- **Python** (`.launch.py`) → most flexible (logic, conditions, loops)
- **XML** (`.launch.xml`) → structured, less logic
- **YAML** (`.launch.yaml`) → compact configs, limited logic
- Docs: [Launch formats]

Why `.launch.py` is the most common

Real robot bringup needs conditional logic.

- Example needs: `use_rviz:=true`, choose `slam:=true/false`, pick map file…
- Python launch can: declare args, include other launch files, use if/else
- You can list launch arguments with:
- ```
  ros2 launch <pkg> <file>.launch.py --show-args
  ```
- Ref: [Using substitutions / show args]

uOttawa
  ```

Minimal `.launch.py` (Part 1/2): imports + function

Every Python launch file must return a `LaunchDescription`.

- ```
  from launch import LaunchDescription
  from launch_ros.actions import Node


  def generate_launch_description():
      # we will add nodes in Part 2
      return LaunchDescription([
          # Node(...), Node(...)
      ])
  ```
- Key idea: `LaunchDescription([ ...actions... ])`

uOttawa
  ```

Minimal `.launch.py` (Part 2/2): launching nodes + names

Add nodes with `Node(package=..., executable=...)`.

- ```
  return LaunchDescription([
    Node(
      package='demo_nodes_cpp',
      executable='talker',
      name='talker1',
      output='screen'
    ),
    Node(
      package='demo_nodes_cpp',
      executable='listener',
      name='listener1',
      output='screen'
    ),
  ])
  ```
- This starts two executables and prints their logs in your terminal.

uOttawa
  ```

Passing YAML params from a launch file

This is the most common pattern in SLAM/Nav2 bringup.

- ```
  from launch_ros.actions import Node

  params_file = '/path/to/config/demo_params.yaml'

  Node(
    package='my_pkg',
    executable='my_node',
    name='my_node',
    parameters=[params_file],
    output='screen'
  )
  ```
- Tip: keep configs in your package: `my_pkg/config/*.yaml`

uOttawa
  ```

Make launch/config discoverable after build

If your package is **ament_cmake**:

- Edit `CMakeLists.txt`:
- ```
  install(DIRECTORY launch config
    DESTINATION share/${PROJECT_NAME})
  ```
- Then rebuild from workspace root:
- ```
  cd ~/ros2_ws
  colcon build --symlink-install
  source install/setup.bash
  ```

If your package is **ament_python**:

- In `setup.py` add data_files (concept):
- ```
  data_files=[
   ('share/my_pkg/launch', ['launch/demo.launch.py']
   ('share/my_pkg/config', ['config/demo_params.yaml
  ]
  ```
- Rule: launch + config must be installed to `share/<pkg>/...`

uOttawa
  ```

Quiz: What does `ros__parameters` do?

In a ROS 2 YAML file, the key `ros__parameters:` is used to...

- **A** Declare a new ROS topic
- **B** Start the parameter section for a node
- **C** Enable TF publishing
- **D** Set QoS history depth

Checklist (you should be able to do this alone)

- Find node + inspect params: `ros2 node list` → `ros2 param list/get/describe`
- Set live (if allowed): `ros2 param set /node_name key value`
- Save a snapshot: `ros2 param dump /node_name > tuned.yaml`
- Write YAML correctly: `<node_name>: ros__parameters: ...` and `/**` wildcard
- Create a launch: `generate_launch_description()` → add `Node(...)` actions
- Pass YAML from launch: `parameters=[params_file]`

uOttawa